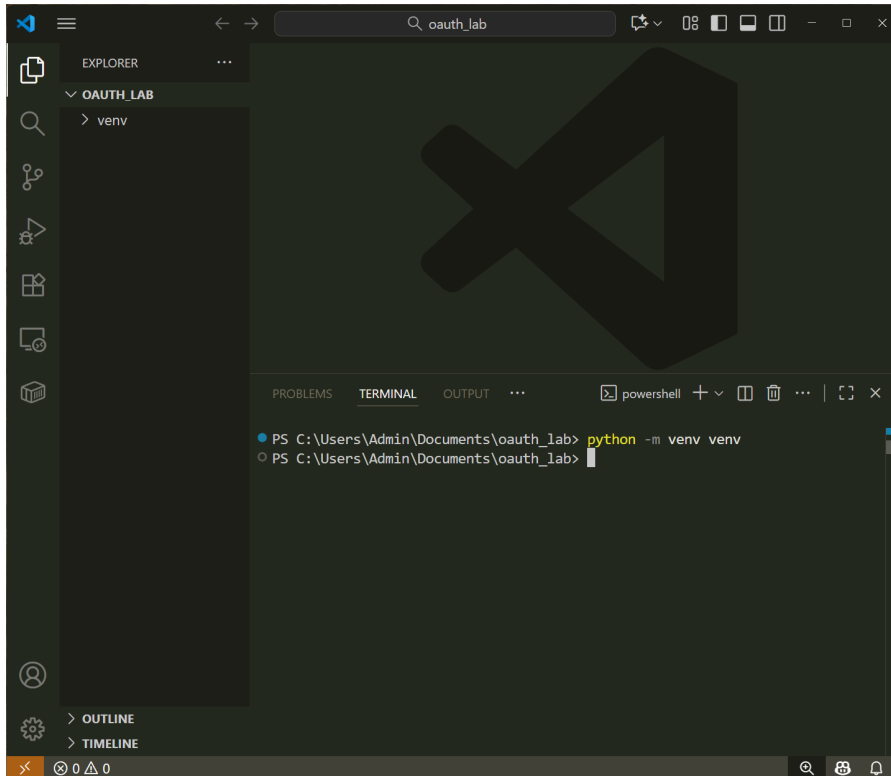
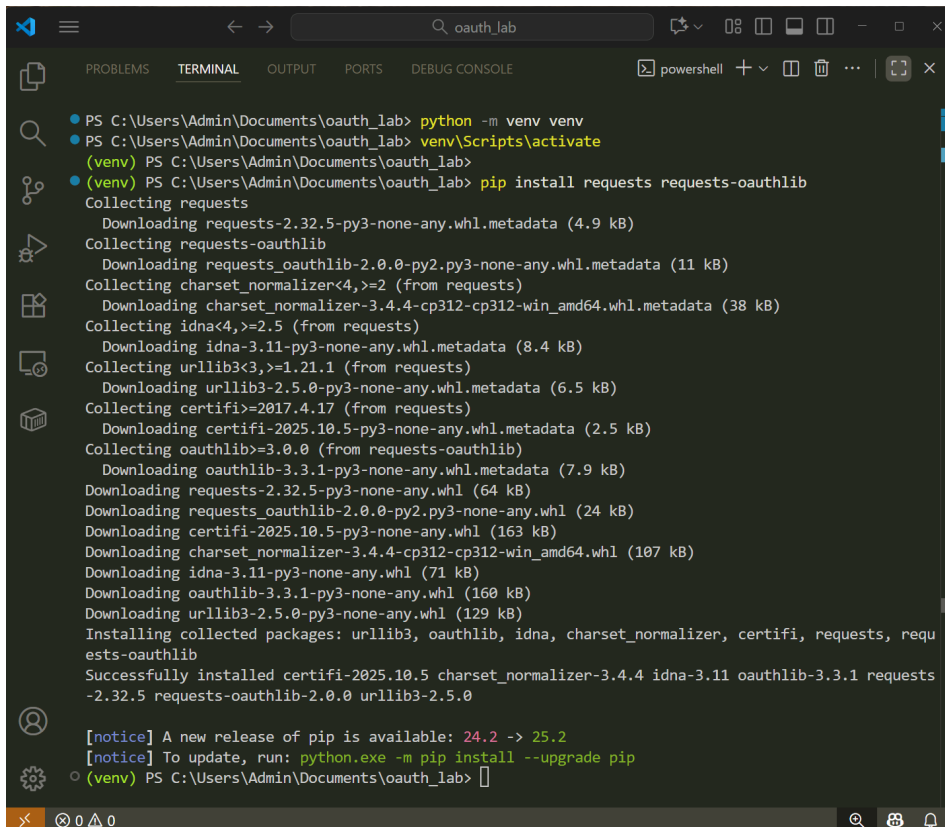


Лабораторная работа: практика OAuth 2.0 в Python

1. Создание виртуального окружения



2. Активация виртуального окружения и установка библиотек



3. Список библиотек

```
(venv) PS C:\Users\Admin\Documents\oauth_lab> pip list
Package            Version
-----
certifi            2025.10.5
charset-normalizer 3.4.4
idna               3.11
oauthlib           3.3.1
pip               24.2
requests          2.32.5
requests-oauthlib 2.0.0
urllib3           2.5.0
```

4. Подготовка файла для переменных окружения

```
.env
1 CLIENT_ID=
2 CLIENT_SECRET=
```

5. Настройка проекта в Google Cloud Console

Создание экрана согласия

Google Cloud OAuth2-Python-Lab Search (/) for resources, docs, products, and more Search

Google Auth Platform / Overview / Create branding

Overview Branding Audience Clients Data Access Verification Center Settings

Project configuration

1 App Information

2 Audience

☐ Internal ⓘ

Only available to users within your organization. You will not need to submit your app for verification. [Learn more about user type](#)

☒ External ⓘ

Available to any test user with a Google Account. Your app will start in testing mode and will only be available to users you add to the list of test users. Once your app is ready to push to production, you may need to verify your app. [Learn more about user type](#)

Next

3 Contact Information

4 Finish

Create Cancel

Добавление Test User

Баранов Д.А. ИБТ 2.1

Google Cloud

OAuth2-Python-Lab

Search (/) for resources, docs, products, and more

Search

Google Auth Platform / Audience

Overview

Branding

Audience

Clients

Data Access

Verification Center

Settings

Audience

Publishing status ⓘ

Testing

Publish app

User type

External ⓘ

Make internal

OAuth user cap ⓘ

While publishing status is set to "Testing", only test users are able to access the app. Allowed user cap prior to app verification is 100, and is counted over the entire lifetime of the app. [Learn more](#)

1 user (1 test, 0 other) / 100 user cap ⓘ

Test users

+ Add users

Filter Enter property name or value ⓘ

User information

dmitriy@gmail.com

Создание OAuth 2.0 Client ID

Google Cloud

OAuth2-Python-Lab

Search (/) for resources, docs, products, and more

Search

Google Auth Platform / Clients / Create client

Overview

Branding

Audience

Clients

Data Access

Verification Center

Settings

Create OAuth client ID

A client ID is used to identify a single app to Google's OAuth servers. If your app runs on multiple platforms, each will need its own client ID. See [Setting up OAuth 2.0](#) for more information. [Learn more](#) about OAuth client types.

Application type *
Web application

Name *
Web client 1

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Authorized JavaScript origins ⓘ

For use with requests from a browser

+ Add URI

Authorized redirect URIs ⓘ

For use with requests from a web server

URIs 1 *
http://localhost:8000/callback

+ Add URI

Note: It may take 5 minutes to a few hours for settings to take effect

Create Cancel

Баранов Д.А. ИБТ 2.1

OAuth client created

The client ID can always be accessed from Clients tab under Google Auth Platform.

i OAuth access is restricted to the [test users](#) listed on your [OAuth consent screen](#)

Client ID 
ps.googleusercontent.com

⚠ Starting in June 2025, you will no longer be able to view or download the client secret once you close this dialog. Make sure you have copied or downloaded the information below and securely stored it.

Client secret 

Creation date October 23, 2025 at 2:40:07 PM GMT+3

Status  Enabled

 Download JSON

OK

Записать ключи в .env

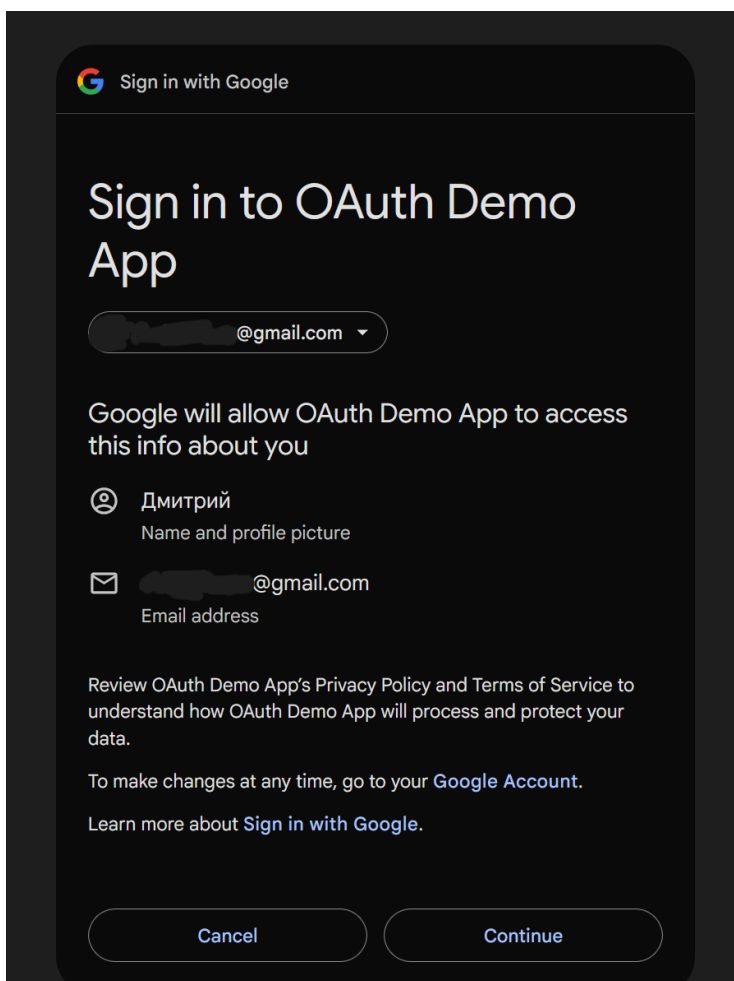
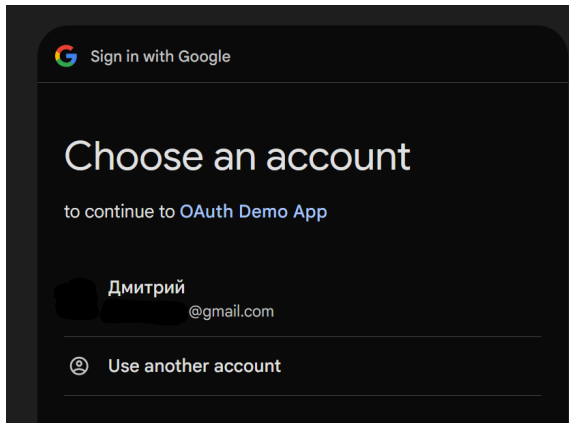
6. Создание google_oauth.py

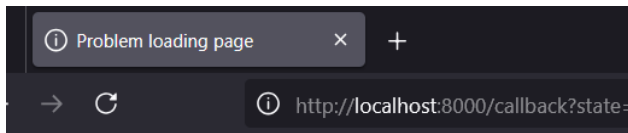
```
google_oauth.py 1 x
google_oauth.py > ...
1 import os
2 from requests_oauthlib import OAuth2Session
3 from dotenv import load_dotenv
4
5 load_dotenv()
6
7 CLIENT_ID = os.getenv("CLIENT_ID")
8 CLIENT_SECRET = os.getenv("CLIENT_SECRET")
9 REDIRECT_URI = "http://localhost:8000/callback"
10
11 SCOPE = ["openid", "email", "profile"]
12
13 AUTHORIZATION_BASE_URL = "https://accounts.google.com/o/oauth2/auth"
14 TOKEN_URL = "https://oauth2.googleapis.com/token"
15
16 oauth = OAuth2Session(CLIENT_ID, redirect_uri=REDIRECT_URI, scope=SCOPE)
17
18 authorization_url, state = oauth.authorization_url(AUTHORIZATION_BASE_URL, access_type="offline", prompt="consent")
19
20 print("Go to the link for authorization:")
21 print(authorization_url)
22
23 redirect_response = input("\nPaste the full redirect URL: ")
24
25 token = oauth.fetch_token(
26     TOKEN_URL,
27     authorization_response=redirect_response,
28     client_secret=CLIENT_SECRET
29 )
30
31 print("\nYour token:")
32 print(token)
33
34 userinfo_endpoint = "https://www.googleapis.com/oauth2/v3/userinfo"
35 r = oauth.get(userinfo_endpoint)
36 print("\nUser information:")
37 print(r.json())
```

7. Запуск скрипта

```
(venv) PS C:\Users\Admin\Documents\oauth_lab> python google_oauth.py
Go to the link for authorization:
https://accounts.google.com/o/oauth2/auth?response_type=code&client_id=
[REDACTED].apps.googleusercontent.com&redirect_uri=
[REDACTED]localhost[REDACTED]&scope=openid+email+profile&state=
[REDACTED]&access_type=offline&prompt=consent

Paste the full redirect URL: [ ]
```





Добавление разрешения на HTTP

```
os.environ['OAUTHLIB_INSECURE_TRANSPORT'] = '1'
```

Обновление вида ссылки

```
AUTHORIZATION_BASE_URL = "https://accounts.google.com/o/oauth2/v2/auth"
```

8. Результат

```
Your token:
{
  'access_token': [REDACTED],
  'expires_in': 3599,
  'refresh_token': [REDACTED],
  'scope': 'https://www.googleapis.com/auth/userinfo.email',
  'token_type': 'Bearer',
  'id_token': [REDACTED]
}

User information:
{
  "sub": "[REDACTED]",
  "name": "Дмитрий Баранов",
  "given_name": "Дмитрий",
  "family_name": "Баранов",
  "picture": "https://lh3.googleusercontent.com/a-/AOh14Gh[REDACTED]",
  "email": "dmitry[REDACTED]@gmail.com",
  "email_verified": true,
  "locale": "ru"
}
```

Задание 2: Обновление токена (refresh token)

9. Модификация кода

```
google_oauth.py X
google_oauth.py > obtain_new_token
1  import os
2  import json
3  from requests_oauthlib import OAuth2Session
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  CLIENT_ID = os.getenv("CLIENT_ID")
9  CLIENT_SECRET = os.getenv("CLIENT_SECRET")
10 REDIRECT_URI = "http://localhost:8000/callback"
11
12 os.environ['OAUTHLIB_INSECURE_TRANSPORT'] = '1'
13
14 SCOPE = ["openid", "email", "profile"]
15 AUTHORIZATION_BASE_URL = "https://accounts.google.com/o/oauth2/v2/auth"
16 TOKEN_URL = "https://oauth2.googleapis.com/token"
17
18 TOKEN_FILE = "token.json"
19
20 def save_token(token_dict):
21     with open(TOKEN_FILE, "w") as f:
22         json.dump(token_dict, f)
23
24 def load_token():
25     if os.path.exists(TOKEN_FILE):
26         with open(TOKEN_FILE, "r") as f:
27             return json.load(f)
28     return None
29
30 def refresh_token(old_token):
31     data = {
32         "grant_type": "refresh_token",
33         "refresh_token": old_token["refresh_token"],
34         "client_id": CLIENT_ID,
35         "client_secret": CLIENT_SECRET
36     }
37     import requests
38     resp = requests.post(TOKEN_URL, data=data)
39     resp.raise_for_status()
40     new_tokens = resp.json()
41
42     old_token.update({
43         "access_token": new_tokens["access_token"],
44         "expires_in": new_tokens.get("expires_in"),
45         "scope": new_tokens.get("scope"),
46         "token_type": new_tokens.get("token_type"),
47     })
48     save_token(old_token)
49     return old_token
50
51 def obtain_new_token():
52     oauth = OAuth2Session(CLIENT_ID, redirect_uri=REDIRECT_URI, scope=SCOPE)
53     authorization_url, state = oauth.authorization_url(
54         AUTHORIZATION_BASE_URL,
55         access_type="offline",
```

```
56     prompt="consent"
57 )
58
59 print("Go to the link for authorization:")
60 print(authorization_url)
61 redirect_response = input("\nPaste the full redirect URL: ")
62 token = oauth.fetch_token(
63     TOKEN_URL,
64     authorization_response=redirect_response,
65     client_secret=CLIENT_SECRET
66 )
67 save_token(token)
68 return token
69
70 def main():
71     token = load_token()
72     if token:
73         if "refresh_token" in token:
74             print("Existing token found, refreshing ...")
75             token = refresh_token(token)
76         else:
77             print("Token found but no refresh_token - obtaining new token ...")
78             token = obtain_new_token()
79     else:
80         print("No token found - obtaining new token ...")
81         token = obtain_new_token()
82
83     print("\nYour token:")
84     print(token)
85
86     oauth = OAuth2Session(CLIENT_ID, token=token)
87     userinfo_endpoint = "https://www.googleapis.com/oauth2/v3/userinfo"
88     r = oauth.get(userinfo_endpoint)
89     print("\nUser information:")
90     print(r.json())
91
92 if __name__ == "__main__":
93     main()
```

10. Результат:

```
{
  "access_token": " ",
  "expires_in": 3599,
  "refresh_token": " ",
  "scope": "https://www.googleapis.com/auth/us",
  "token_type": "Bearer",
  "id_token": " "
}
```